ECEC661 — Homework 1

Gary Pham — gp492

April 8, 2026

Abstract

This report presents the design, implementation, and verification of three digital circuits described in VHDL for ECEC661 Homework 1: (1) a combinational upper/lower output limiter (Problem 2.6.12), (2) a sequential synchronous palindrome checker (Problem 3.5.11), and (3) an accumulator built around a Vivado IP core (Problem 7.5.3). Each design is accompanied by a self-checking testbench that applies stimulus vectors and automatically verifies expected outputs via VHDL `assert` statements. Waveform results are captured from Xilinx XSim [1] simulation in batch mode through VCD export and plotted with Python. All simulations pass with zero assertion errors.

1 Introduction

Hardware description languages such as VHDL [2] enable designers to specify digital circuits at the register-transfer level (RTL) and verify them through simulation before committing to silicon or FPGA implementation. This report documents three exercises that cover a range of fundamental HDL design patterns:

- **Combinational logic** — the output limiter uses a purely combinational process with unsigned comparisons to clamp an input within a parameterised range (Section 3).
- **Synchronous sequential logic** — the palindrome checker uses clocked shift registers and a bit-reversal comparison to determine whether two serially received bit-streams form a palindrome pair (Section 4).
- **IP integration** — the accumulator wraps a Vivado Accumulator IP core (or a portable behavioural stand-in) within a simple top-level entity, demonstrating hierarchical design and IP reuse (Section 5).

All designs target Xilinx Vivado 2025.2 [1] and are simulated with the included XSim simulator. Section 2 describes the simulation and verification methodology, followed by dedicated sections for each design. Section 6 summarises the results.

2 Simulation and Verification Methodology

Each design is paired with a self-checking VHDL testbench. The testbenches share a common verification strategy:

1. **Stimulus generation.** The testbench drives input signals according to a predefined sequence (either exhaustive or selected corner cases).
2. **Automated checking.** After each stimulus application (and appropriate settling or clock-edge wait), a VHDL `assert` statement compares the DUT output against the analytically expected value. Any mismatch raises an `error`-severity message.
3. **Pass/fail reporting.** Upon completing all vectors without error, the testbench prints a "`<testbench_name> PASSED`" note and terminates via `std.env.stop`.

Waveforms are captured in batch mode (no GUI) by elaborating with `xelab ... -debug typical` to enable trace information, then running XSim with a Tcl script that calls `open_vcd`, `log_vcd`, and `close_vcd` [1]. The resulting IEEE 1364 VCD files [3] are parsed and plotted with Python (`vcdvcd` and `matplotlib`). Table 1 lists the deliverables for each problem.

Table 1: File manifest for each homework problem.

Problem	Design Source	Testbench	Waveform
2.6.12 Output Limiter	`output_limiter.vhd`	`tb_output_limiter.vhd`	`waveform_limiter.png`
3.5.11 Palindrome Checker	`palindrome_synch_ckt.vhd`	`tb_palindrome_synch_ckt.vhd`	`waveform_palindrome.png`
7.5.3 Accumulator IP	`accumulator_top.vhd`, `c_accum_0.vhd`	`tb_accumulator_top.vhd`	`waveform_accumulator.png`

3 Problem 2.6.12 — Upper and Lower Output Limiter

3.1 Problem Statement

Design a combinational circuit that receives an n-bit unsigned input x and produces an output z clamped between a lower limit l and an upper limit u. Both limits are configurable via VHDL generics. Formally:

$$z = \begin{cases} l & \text{if } x < l, \\ u & \text{if } x > u, \\ x & \text{otherwise.} \end{cases} \tag{1}$$

3.2 Design Description

The implementation consists of a package `output_limiter` that declares the bit-width constant $n = 4$, and an entity `user_logic` parameterised by generics `u` and `l` (both n-bit vectors, defaulting to all-ones and all-zeros, respectively). The architecture contains a single sensitivity-list process that evaluates the unsigned comparisons in Equation (1) using the `IEEE.NUMERIC_STD` `unsigned` type [2].

Since the process is purely combinational (the sensitivity list contains only the input `x` and there are no clock or storage elements), synthesis tools will infer a comparator–multiplexer chain with no registers.

3.3 Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

package output_limiter is
  constant n : natural := 4;
end package output_limiter;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.output_limiter.all;

entity user_logic is
  generic(
    u : std_logic_vector(n - 1 downto 0) := (others => '1');
    l : std_logic_vector(n - 1 downto 0) := (others => '0')
  );
  port(
    x : in  std_logic_vector(n - 1 downto 0);
    z : out std_logic_vector(n - 1 downto 0)
  );
end entity user_logic;

architecture rtl of user_logic is
begin
  process(x)
  begin
    if unsigned(x) < unsigned(l) then
      z <= l;
    elsif unsigned(x) > unsigned(u) then
      z <= u;
    else
      z <= x;
    end if;
  end process;
end architecture rtl;
```

Listing 1: output_limiter.vhd — Package and entity for the output limiter.

3.4 Testbench Design

The testbench `tb_output_limiter` instantiates `user_logic` with $u =$ `"1010"` (decimal 10) and $l =$ `"0101"` (decimal 5). It performs an *exhaustive* sweep of all $2^n = 16$ input values ($x = 0, 1, \ldots, 15$) with a 10 ns settling delay between each. At every step, the expected output is computed in a VHDL variable and compared against the DUT output `z` using an `assert` statement.

The exhaustive approach guarantees that every possible input combination is tested — a practical strategy for small bit-widths. For larger designs, a corner-case strategy would be more appropriate.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use std.env.all;
use work.output_limiter.all;

entity tb_output_limiter is
end entity tb_output_limiter;

architecture sim of tb_output_limiter is
  signal x : std_logic_vector(n - 1 downto 0) := (others => '0');
  signal z : std_logic_vector(n - 1 downto 0);
begin
  dut : entity work.user_logic
    generic map(
      u => "1010", -- 10
      l => "0101"  -- 5
    )
    port map(
      x => x,
      z => z
    );

  stim : process
    variable expected : unsigned(n - 1 downto 0);
  begin
    for i in 0 to (2 ** n) - 1 loop
      x <= std_logic_vector(to_unsigned(i, n));
      wait for 10 ns;

      if i < 5 then
        expected := to_unsigned(5, n);
      elsif i > 10 then
        expected := to_unsigned(10, n);
      else
        expected := to_unsigned(i, n);
      end if;

      assert unsigned(z) = expected
        report "Limiter mismatch at x=" & integer'image(i)
        severity error;
    end loop;

    report "tb_output_limiter PASSED" severity note;
    stop;
    wait;
  end process;
end architecture sim;
```

Listing 2: tb_output_limiter.vhd — Exhaustive self-checking testbench.

3.5 Simulation Waveform

Figure 1: Output Limiter waveform: `x` sweeps 0–15 while `z` is clamped between 5 and 10.

3.6 Results and Discussion

Table 2 summarises the observed clamping behaviour across the three distinct input regions.

Table 2: Output limiter verification summary ($l = 5$, $u = 10$).

Input range	**Expected z**	**Observed behaviour**
$x \in [0, 4]$	5 (lower limit)	z remains constant at 5
$x \in [5, 10]$	x (pass-through)	z tracks x: 5, 6, 7, 8, 9, 10
$x \in [11, 15]$	10 (upper limit)	z remains constant at 10

The simulation log confirms: `tb_output_limiter PASSED` with zero assertion errors, verifying functional correctness for every possible 4-bit input.

4 Problem 3.5.11 — Sequential Palindrome Checker

4.1 Problem Statement

Design a synchronous sequential circuit that receives two serial bit-streams x and y (one bit per clock cycle, LSB first) and, after n clock cycles, asserts output z if the accumulated x vector equals the bit-reversal of the accumulated y vector:

$$z = \begin{cases} 1 & \text{if } X[n{-}1:0] = \text{reverse}(Y[n{-}1:0]), \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

4.2 Design Description

Entity `palindrome_synch_ckt` is parameterised by generic n (default 5). Internally, two n-bit shift registers (`x_shift`, `y_shift`) and a counter `count` track how many bits have been captured since the last reset. On each rising clock edge:

- If `reset = '1'`: all registers and the output are cleared to zero.
- If `count` $<$ `n`: the current input bits are shifted into the LSB of the respective registers, and `count` is incremented.

- When `count` reaches $n-1$ (i.e. the n-th sample is being captured in the same clock cycle): the comparison `x_next` $\stackrel{?}{=}$ reverse(`y_next`) is evaluated on the *about-to-be-latched* values via local variables, ensuring the result is available one clock edge after the last input.

A helper function `reverse_bits` performs a compile-time-known loop reversal of a `std_logic_vector`, which synthesises to wire re-ordering (zero logic cost) [5].

4.3 Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity palindrome_synch_ckt is
  generic(n : natural := 5);
  port(
    x, y, ck, reset : in  std_logic;
    z               : out std_logic
  );
end entity palindrome_synch_ckt;

architecture rtl of palindrome_synch_ckt is
  signal x_shift : std_logic_vector(n - 1 downto 0) := (others => '0');
  signal y_shift : std_logic_vector(n - 1 downto 0) := (others => '0');
  signal count   : natural range 0 to n := 0;
  signal z_reg   : std_logic := '0';

  function reverse_bits(v : std_logic_vector) return std_logic_vector is
    variable r : std_logic_vector(v'range);
  begin
    for i in 0 to v'length - 1 loop
      r(i) := v(v'length - 1 - i);
    end loop;
    return r;
  end function;
begin
  z <= z_reg;

  process(ck)
    variable x_next : std_logic_vector(n - 1 downto 0);
    variable y_next : std_logic_vector(n - 1 downto 0);
  begin
    if rising_edge(ck) then
      if reset = '1' then
        x_shift <= (others => '0');
        y_shift <= (others => '0');
        count   <= 0;
        z_reg   <= '0';
      else
        if count < n then
          x_next := x_shift(n - 2 downto 0) & x;
```

```
          y_next := y_shift(n - 2 downto 0) & y;

          x_shift <= x_next;
          y_shift <= y_next;
          count   <= count + 1;

          -- Evaluate exactly after the n-th sample is captured.
          if count = n - 1 then
            if x_next = reverse_bits(y_next) then
              z_reg <= '1';
            else
              z_reg <= '0';
            end if;
          end if;
        else
          if x_shift = reverse_bits(y_shift) then
            z_reg <= '1';
          else
            z_reg <= '0';
          end if;
        end if;
      end if;
    end if;
  end process;
end architecture rtl;
```

Listing 3: palindrome_synch_ckt.vhd — Synchronous palindrome checker.

4.4 Testbench Design

The testbench `tb_palindrome_synch_ckt` exercises two back-to-back scenarios with $n = 5$, separated by a reset pulse:

Test 1 — Palindrome match.

- x sequence: 1, 0, 1, 0, 1 $\Rightarrow$ $X = \texttt{10101}$.
- y sequence: 1, 0, 1, 0, 1 $\Rightarrow$ $Y = \texttt{10101}$.
- $\text{reverse}(Y) = \texttt{10101} = X$, hence z should be `'1'`.

Test 2 — Non-palindrome.

- x sequence: 1, 1, 0, 0, 1 $\Rightarrow$ $X = \texttt{11001}$.
- y sequence: 1, 0, 1, 0, 0 $\Rightarrow$ $Y = \texttt{10100}$.
- $\text{reverse}(Y) = \texttt{00101} \neq X$, hence z should be `'0'`.

The testbench asserts the expected z value one clock cycle after the final input of each test, then reports pass/fail.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use std.env.all;

entity tb_palindrome_synch_ckt is
```

```
end entity tb_palindrome_synch_ckt;

architecture sim of tb_palindrome_synch_ckt is
  constant n_c : natural := 5;
  signal x     : std_logic := '0';
  signal y     : std_logic := '0';
  signal ck    : std_logic := '0';
  signal reset : std_logic := '1';
  signal z     : std_logic;

  constant Tck : time := 10 ns;
begin
  dut : entity work.palindrome_synch_ckt
    generic map(n => n_c)
    port map(
      x => x,
      y => y,
      ck => ck,
      reset => reset,
      z => z
    );

  ck <= not ck after Tck / 2;

  stim : process
    -- Test 1: x sequence 1,0,1,0,1 and y sequence 1,0,1,0,1
    -- reverse(y) = 1,0,1,0,1 -> palindrome match -> z = 1
    type bit_array_t is array (0 to n_c - 1) of std_logic;
    constant x_seq_match : bit_array_t := ('1', '0', '1', '0', '1');
    constant y_seq_match : bit_array_t := ('1', '0', '1', '0', '1');

    -- Test 2: x sequence 1,1,0,0,1 and y sequence 1,0,1,0,0
    -- reverse(y) = 0,0,1,0,1 /= x -> no match -> z = 0
    constant x_seq_miss : bit_array_t := ('1', '1', '0', '0', '1');
    constant y_seq_miss : bit_array_t := ('1', '0', '1', '0', '0');
  begin
    reset <= '1';
    wait for 2 * Tck;
    reset <= '0';

    for i in 0 to n_c - 1 loop
      x <= x_seq_match(i);
      y <= y_seq_match(i);
      wait until rising_edge(ck);
    end loop;

    wait until rising_edge(ck);
    assert z = '1'
      report "Palindrome test 1 failed (expected z=1)"
      severity error;

    reset <= '1';
    wait until rising_edge(ck);
    reset <= '0';
```

```
    for i in 0 to n_c - 1 loop
      x <= x_seq_miss(i);
      y <= y_seq_miss(i);
      wait until rising_edge(ck);
    end loop;

    wait until rising_edge(ck);
    assert z = '0'
      report "Palindrome test 2 failed (expected z=0)"
      severity error;

    report "tb_palindrome_synch_ckt PASSED" severity note;
    stop;
    wait;
  end process;
end architecture sim;
```

Listing 4: tb_palindrome_synch_ckt.vhd — Two-case self-checking testbench.

4.5 Simulation Waveform

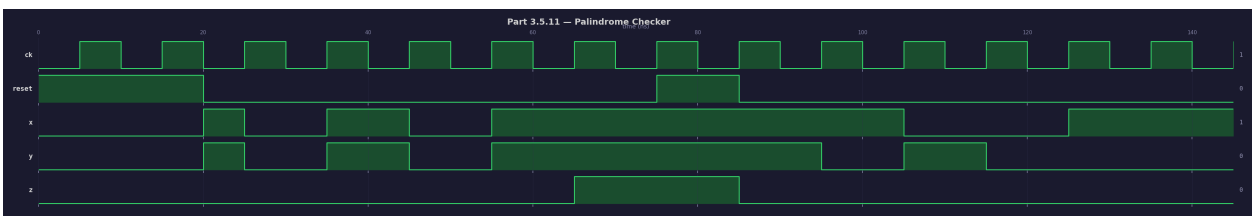

Figure 2: Palindrome Checker waveform: Test 1 produces $z = 1$ (palindrome match); Test 2 produces $z = 0$ (no match).

4.6 Results and Discussion

The waveform in Figure 2 shows two distinct test windows separated by a reset pulse:

1. **First window ($t \approx 20$–70 ns):** After five clock cycles of input capture, `z` rises to `'1'`, confirming the palindrome match between $X =$ `10101` and reverse$(Y) =$ `10101`.
2. **Second window ($t \approx 80$–130 ns):** After five more clock cycles with different inputs, `z` remains at `'0'`, confirming the non-palindrome case where $X =$ `11001` differs from reverse$(Y) =$ `00101`.

The simulation log reports `tb_palindrome_synch_ckt PASSED` with zero assertion errors.

5 Problem 7.5.3 — Accumulator IP and Testbench

5.1 Problem Statement

Using the Vivado IP Catalog, generate an Accumulator IP core configured for 4-bit signed input and 8-bit signed output with synchronous clear. Wrap the IP in a top-level entity and

write a testbench that verifies correct accumulation over a sequence of signed inputs.

5.2 Design Description

The design is structured as a two-level hierarchy:

1. `accumulator_top` — a thin RTL wrapper that exposes ports `ck` (clock), `sclr` (synchronous clear), `x` (4-bit signed input), and `q` (8-bit signed output). It instantiates the accumulator IP via direct entity instantiation.
2. `c_accum_0` — the accumulator core. Two interchangeable implementations exist:
   - *Vivado IP:* Generated from the IP Catalog (`Accumulator` category); configured for 4-bit signed input, 8-bit signed output, with synchronous clear [4].
   - *Behavioural stand-in:* A portable VHDL model (`c_accum_0.vhd`) with identical port names (`B`, `CLK`, `SCLR`, `Q`) and identical semantics. This allows simulation without requiring IP generation.

On each rising clock edge: if `SCLR` = 1, the internal accumulator register resets to zero; otherwise, the 4-bit signed input is sign-extended to 8 bits and added to the running sum:

$$Q^{(k)} = \begin{cases} 0 & \text{if SCLR} = 1, \\ Q^{(k-1)} + \text{sign_ext}(B, 8) & \text{otherwise.} \end{cases} \tag{3}$$

5.3 Source Code

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity accumulator_top is
  port(
    ck    : in  std_logic;
    sclr  : in  std_logic;
    x     : in  std_logic_vector(3 downto 0);
    q     : out std_logic_vector(7 downto 0)
  );
end entity accumulator_top;

architecture rtl of accumulator_top is
begin
  u_ip : entity work.c_accum_0
    port map(
      B    => x,
      CLK  => ck,
      SCLR => sclr,
      Q    => q
    );
end architecture rtl;
```

Listing 5: accumulator_top.vhd — Top-level IP wrapper.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```vhdl
use IEEE.NUMERIC_STD.ALL;

-- Behavioral stand-in for Vivado C_ACCUM IP (4-bit signed in, 8-bit
    signed out).
entity c_accum_0 is
  port(
    B    : in  std_logic_vector(3 downto 0);
    CLK  : in  std_logic;
    SCLR : in  std_logic;
    Q    : out std_logic_vector(7 downto 0)
  );
end entity c_accum_0;

architecture rtl of c_accum_0 is
  signal acc : signed(7 downto 0) := (others => '0');
begin
  process(CLK)
  begin
    if rising_edge(CLK) then
      if SCLR = '1' then
        acc <= (others => '0');
      else
        acc <= acc + resize(signed(B), 8);
      end if;
    end if;
  end process;

  Q <= std_logic_vector(acc);
end architecture rtl;
```

Listing 6: c_accum_0.vhd — Behavioural stand-in for the Vivado Accumulator IP.

5.4 Testbench Design

The testbench `tb_accumulator_top` applies a sequence of 20 signed 4-bit values spanning the full representable range $[-8, +7]$ plus selected mixed-sign values:

```
-8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, -8, 7, -1, 6
```

The verification procedure is:

1. Assert synchronous clear: pulse `sclr` high for one clock cycle, then verify $q = 0$.
2. For each of the 20 input samples: drive `x`, wait for one rising clock edge plus 1 ns propagation, then assert that `q` equals the analytically computed running sum using a VHDL variable `expected_sum`.
3. Report `PASSED` upon completion.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use std.env.all;

entity tb_accumulator_top is
```

```
end entity tb_accumulator_top;

architecture sim of tb_accumulator_top is
  signal ck   : std_logic := '0';
  signal sclr : std_logic := '0';
  signal x    : std_logic_vector(3 downto 0) := (others => '0');
  signal q    : std_logic_vector(7 downto 0);

  constant Tck : time := 10 ns;

  type int_array_t is array (natural range <>) of integer;
  -- 20 signed 4-bit values in allowed range [-8..7]
  constant seq20 : int_array_t := (
    -8, -7, -6, -5, -4, -3, -2, -1, 0, 1,
     2,  3,  4,  5,  6,  7, -8,  7, -1, 6
  );
begin
  dut : entity work.accumulator_top
    port map(
      ck => ck,
      sclr => sclr,
      x => x,
      q => q
    );

  ck <= not ck after Tck / 2;

  stim : process
    variable expected_sum : integer := 0;
  begin
    sclr <= '1';
    wait until rising_edge(ck);
    wait for 1 ns;
    sclr <= '0';
    wait until rising_edge(ck);
    wait for 1 ns;
    assert signed(q) = to_signed(0, 8)
      report "Accumulator clear failed"
      severity error;

    for i in seq20'range loop
      x <= std_logic_vector(to_signed(seq20(i), 4));
      wait until rising_edge(ck);
      wait for 1 ns;

      expected_sum := expected_sum + seq20(i);
      assert signed(q) = to_signed(expected_sum, 8)
        report "Accumulator mismatch at sample " & integer'image(i)
        severity error;
    end loop;

    report "tb_accumulator_top PASSED" severity note;
    stop;
    wait;
```

```
  end process;
end architecture sim;
```

Listing 7: tb_accumulator_top.vhd — 20-sample self-checking testbench.

Expected running-sum trace. Table 3 shows the analytically derived running sum at each sample, which the testbench verifies cycle-by-cycle.

Table 3: Accumulator expected running-sum trace (Σ is the cumulative sum after sample #).

#	x	Σ	#	x	Σ	#	x	Σ	#	x	Σ
0	-8	-8	5	-3	-33	10	2	-33	15	7	-8
1	-7	-15	6	-2	-35	11	3	-30	16	-8	-16
2	-6	-21	7	-1	-36	12	4	-26	17	7	-9
3	-5	-26	8	0	-36	13	5	-21	18	-1	-10
4	-4	-30	9	1	-35	14	6	-15	19	6	-4

5.5 Simulation Waveform

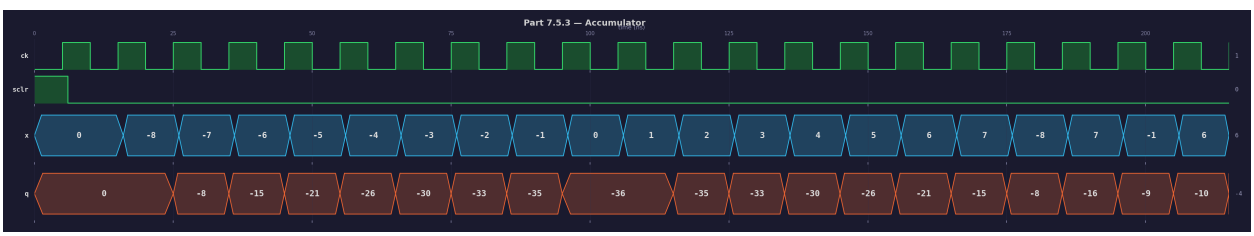

Figure 3: Accumulator waveform: `x` shows each signed 4-bit input; `q` shows the 8-bit running sum matching Table 3.

5.6 Results and Discussion

The waveform in Figure 3 confirms correct operation:

1. **Synchronous clear:** After the initial `sclr` pulse, `q` is zero as expected.
2. **Accumulation accuracy:** The `q` trace matches the expected running sum in Table 3. The sum descends to -36 during samples 0–8 (all negative or zero inputs), then rises as positive values are added during samples 9–15, and finally settles at -4 after the last four mixed-sign inputs.
3. **Signed arithmetic:** The accumulator correctly handles two's complement sign extension from 4 bits to 8 bits (via `resize(signed(B),8)` in the behavioural model) throughout the entire sequence.

The simulation log reports `tb_accumulator_top PASSED` with zero assertion errors across all 20 samples plus the clear check.

5.7 IP Portability Note

On a Vivado-equipped machine, the Vivado IP Catalog `c_accum_0` core can be generated and will seamlessly replace the behavioural `c_accum_0.vhd`. The port interface (`B`, `CLK`, `SCLR`, `Q`) and accumulation semantics are identical [4]. The behavioural stand-in is provided for portable simulation without requiring IP generation.

6 Conclusion

Three VHDL designs were implemented and verified through self-checking testbenches simulated in Xilinx XSim:

- The **output limiter** correctly clamps unsigned inputs within the parameterised bounds, verified exhaustively over all $2^4 = 16$ input values.
- The **palindrome checker** correctly identifies palindrome and non-palindrome serial bit-stream pairs, verified across two representative test cases with reset separation.
- The **accumulator** correctly computes a signed running sum with synchronous clear, verified against 20 analytically derived expected values.

All three testbenches report `PASSED` with zero assertion errors, confirming functional correctness of each design.

References

[1] AMD/Xilinx, "Vivado Design Suite User Guide: Logic Simulation (UG900)," v2025.2, 2025. https://docs.amd.com/r/en-US/ug900-vivado-logic-simulation

[2] IEEE, "IEEE Standard VHDL Language Reference Manual," IEEE Std 1076-2008, 2009.

[3] IEEE, "IEEE Standard for Verilog Hardware Description Language — Value Change Dump (VCD)," IEEE Std 1364-2005, Sec. 18, 2006.

[4] AMD/Xilinx, "Accumulator (PG119)," LogiCORE IP Product Guide, v12.0, 2015. https://docs.amd.com/v/u/en-US/pg119-c-accum

[5] V. A. Pedroni, *Circuit Design and Simulation with VHDL*, 2nd ed. MIT Press, 2010.